

The towers of power

Overview

The overview of our tower defense game is based on the initial project plan we produced at the beginning of the project. By comparing it to the project plan, we can highlight what we succeeded in, and if there are any failures.

The requirements we met are marked with a checkmark. The unmet requirements are marked with a cross, and a comment describing how our finished game differs from the initial requirement.

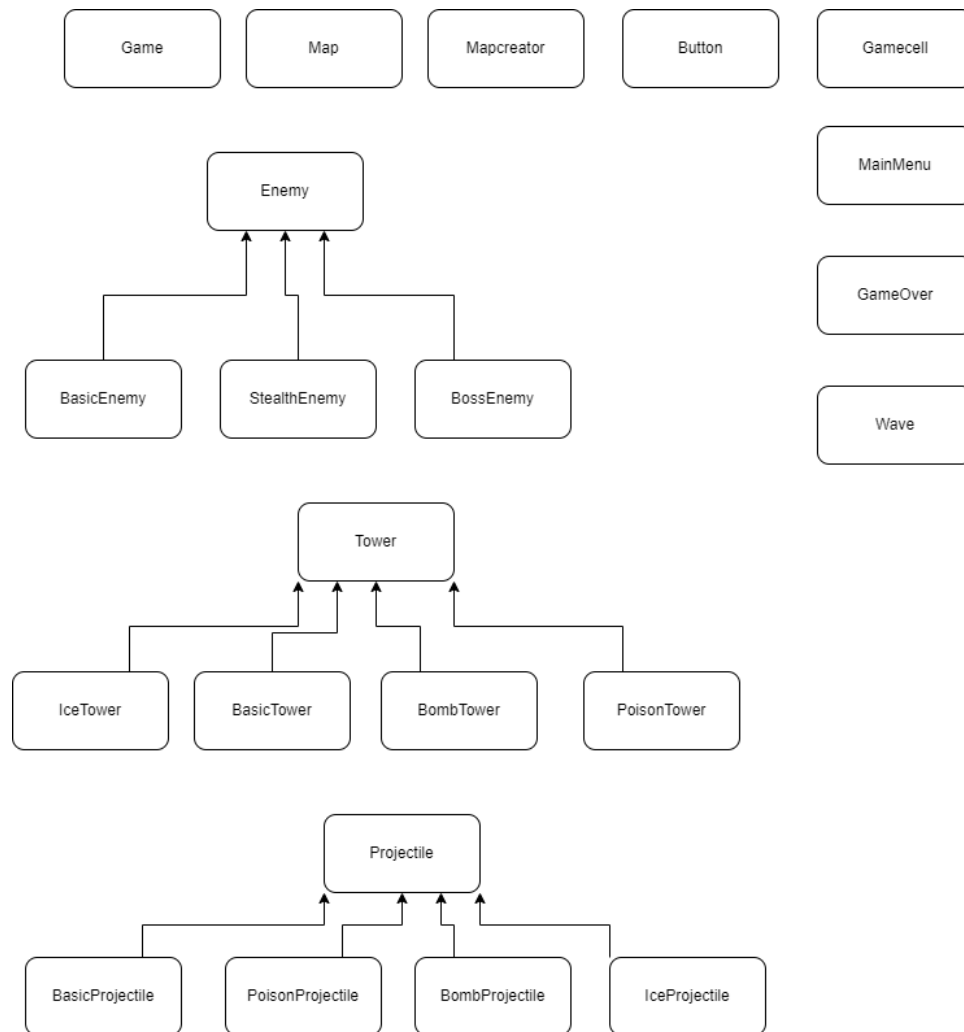
Our initial requirements:

- Maps that are read from a file. ✓
- Towers:
 - Standard tower that damages one enemy at a time. ✓
 - Freeze tower that freezes enemies during a short period of time. ✓
 - Bomb tower with an area of effect damage. ✗ (no AOE)
 - Poison tower with damage over time. ✓
- Enemies:
 - Basic enemy ✓
 - First round hp: 1.
 - Standard speed.
 - Immune/stealthy enemy
 - Not affected by AoE ✗ and damage over time. ✓
 - Squishy and fast. ✓
 - Boss enemy that splits into smaller enemies when killed. ✗ (does not split into multiple enemies. Instead, boss enemy has high hp.)

- The towers are bought with money that the player gets from killing enemies. ✓
- After buying a tower the player will be able to upgrade it for more damage and faster wave clear. ✓
- The enemies' hp increases over time and wave sizes get bigger as the game progresses. ✓
- The game will include 7 different levels with increasing difficulty. ✗ (Only have 5)
- The game will be played with the mouse. ✓
- At least a rudimentary user interface will be implemented to show resources, waves, number of enemies, health, shop, and score. ✓
- There will be sound effects when towers attack and enemies die. ✗ (Only enemy death sounds)
- The purpose of the program is to entertain. ✓✓✓
- When starting the program, the player will be redirected to the main menu with three options: ✓
 - Level editor ✓
 - Level selector/play game ✗ (Only play game)
 - Options ✗
- When playing the game all interactions will be done using the mouse, for example buying new tower, upgrading old ones, or selling towers. ✓

Software structure

The following diagram will show the class relationships in our game, and how our program is structured



Our “main.cpp” file is responsible for switching between different states of the game (e.g., main menu, play game, and map creator). Each state then has its own class responsible for its high-level logic. So, the “Game” class is responsible for the game logic, the MapCreator class is responsible for the map creator logic, and so on.

We then have classes for the five main components of our game; player, enemies, towers, projectiles, and buttons. As can be seen from the diagram, three of them have child classes for different implementations (different enemies, towers, and projectiles).

For more information about our class relationships, and an extensive collection of diagrams, look at our [Doxygen_manual.pdf](#)

External libraries

We only used external libraries for the graphics and audio. In the beginning of the project, we tried to get familiar with QT, but quickly realized SFML is the appropriate choice for us.

- [SFML](#)
 - Graphics and audio

Instructions for building and using the software

Installation:

From project root:

Run the installation shell script:

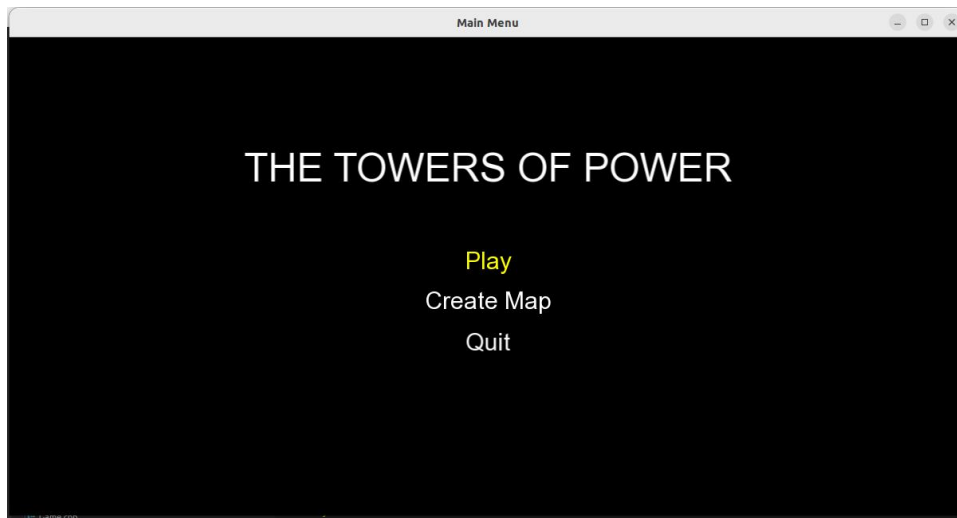
```
$ ./install.sh
```

The binary can now be found in the project root:

```
$ ./tower_defence_game
```

Usage

When running the software, you will be presented with the main menu. From here you have 3 options, “Quit” which will exit the game, “Play” which will initialize the game and “Create map” which will take you to the map creator. The game comes with a pre-defined map, so you can start playing the game immediately if you want to.



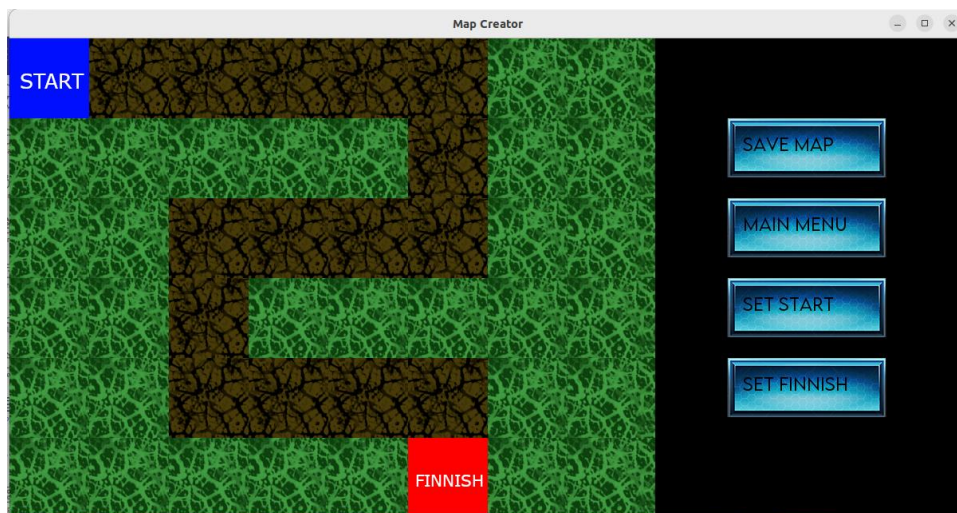
Editing the map

When you open the map creator, you will be greeted by a fully grass filled map. Determining if a tile should be grass or road can be done by clicking on it. You are free to generate the route you want, **as long as the path is only one tile wide.**





Once you are happy with the path, set the start and the finish with the “SET START” and “SET FINISH” buttons.

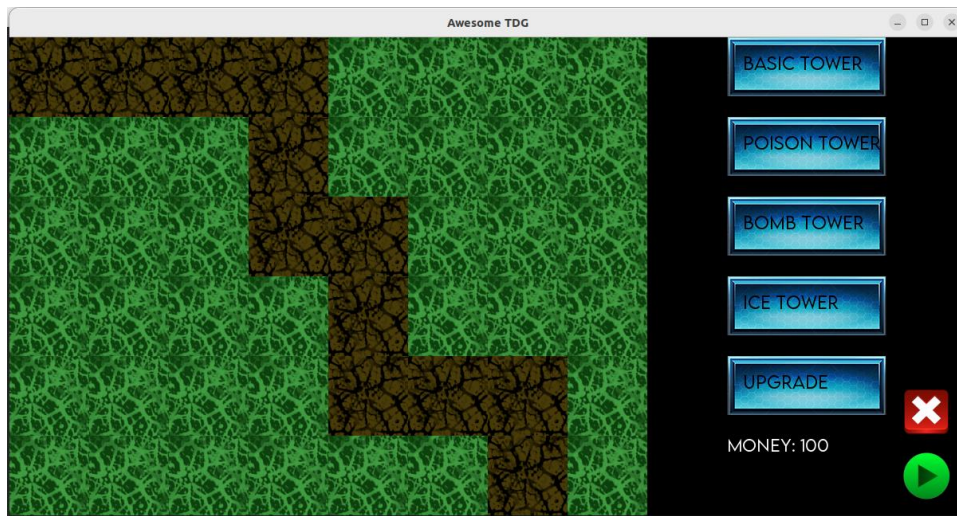


After this, you can press the “SAVE MAP” button to save the map. You can then return to the main menu.

Playing the game

After pressing “Play” in the main menu, the game will open. On the right you see multiple buttons.

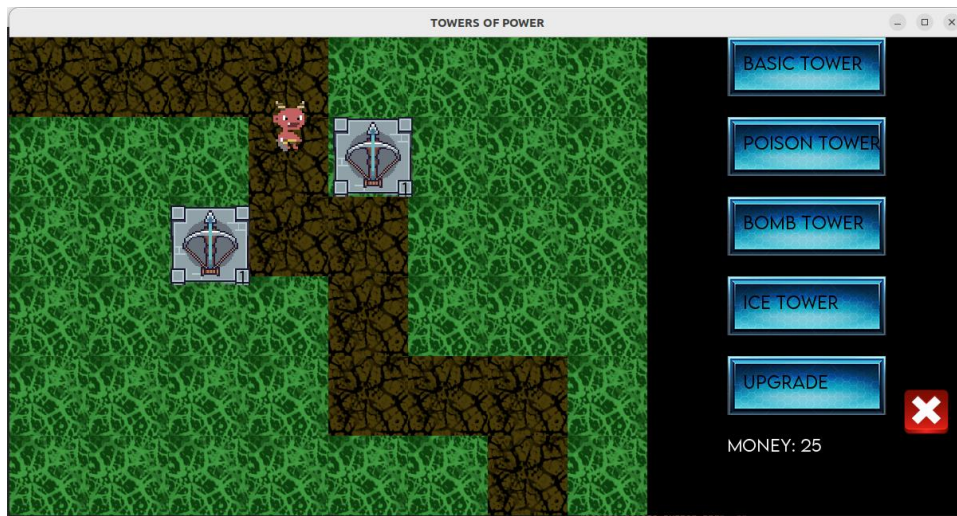
The four top buttons are used for placing towers. Towers are placed by clicking on a tower button, and then clicking on the map. Towers cannot be placed on the route, or on top of another tower. It is possible to place towers both before and during a wave.



After towers have been placed on the map, they can be selected by clicking on them. This will show the effective range of the tower. While a tower is selected, it can be upgraded through the “Upgrade” button or removed by clicking on the red x-button.

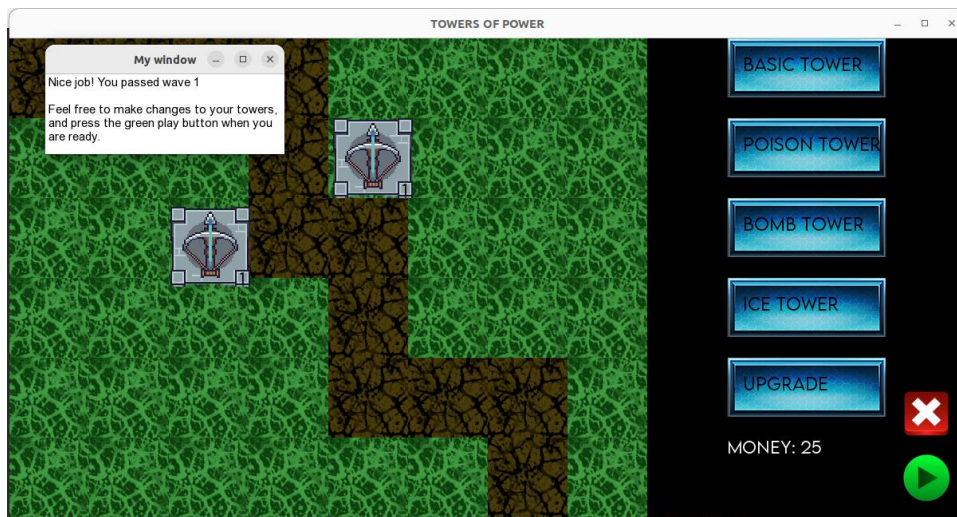


When you are ready for the first wave, you can press the green play button. Enemies will start showing up. You will earn money for every killed enemy, which enables you to buy more towers.



If an enemy makes it to the end of the route, you will lose the game. After losing the game, you will have the option to return to the main menu or quit the game.

If you manage to kill every enemy of the wave, the entire process will start over, but with increased difficulty. For every wave, the towers firing rate increases.



Towers and enemies

The player can purchase 4 different towers with the following attributes:

Basic damage tower:

- Fires projectiles that deal damage. Basic tower with no special attributes
- Damage = 1hp

Poison tower:

- Fires projectiles that deal damage over time to enemies.
- Damage = 2hp/second for a duration of 5 seconds

Bomb tower:

- Fires projectiles that deal more damage, has bigger range and slower attack speed.
- Damage = 3hp

Ice tower:

- Fires projectiles that freeze enemies in place for a short while.
- Freeze time = 2 seconds

In addition, there are 3 different types of enemies:

Basic enemy:

- Standard enemy, moves forward trying to reach the end.
- Starts out with hp = 1. For every round it gains 1hp.
- Spawns every round.

Stealth enemy:

- Like regular enemies but faster. Immune to bomb and poison towers.
- Starts out with hp = 1. For every round it gains 1hp.
- Spawns from round 2 onwards.

Boss enemy:

- Big slow enemies with a lot of health. Immune to freeze towers
- Starts out with hp = 5. For every round it gains 5hp.

- Spawns on rounds 4 and 5.

Testing

For our testing we did not implement any automated testing or unit testing. This is a team decision which we have good reasoning for. As we had to develop this game under a tight schedule, we wanted to prioritize implementing the functionalities we set as requirements, and not put too much effort into writing test scripts.

So how did we make sure our code worked? Most of the implementations were related to the visual components -> testing by seeing if game does what was intended. If we were presented with bugs, these were solved by stepping through the code using the debugger.

Work log

Format: What was done (Time spent)

Week 1:

David: Added external library to repository, created Cmakelist file for compiling, created basic game engine (8h).

Markus: Research on graphical libraries. Configuring makefiles. (6h)

Alfred: Planning structure for tower/enemy parent and child classes. (5h)

Axel: Investigated how to create a map that enemies could navigate through. This led to the decision to split the map into a grid of game tiles (4h).

Week 2:

David: Added enemies, ability to interact with game window, created assets for enemies, projectiles and towers. Added wave object to generate enemies (12h)

Markus: Implemented enemy movement and tower placement. Setup doxygen. (10h)

Alfred: Added basic implementation of tower and enemy parent class. (6h)

Axel: Implemented the GameCell and Map class (6h)

Week 3:

David: Added hitboxes for enemies, updated pathing to use nodes contained in a linked list, added collision detection for hitboxes, proof of concept of projectile spawning and movement (14h).

Markus: Restricted tower placement, algorithm for enemy movement, pathfinding (12h)

Alfred: Added implementation for basic tower and basic enemy, testing projectiles and movement. (7h)

Axel: Worked on how to draw the map in the SFML window and implemented non hardcoded maps. (10h)

Week 4:

David: Added buttons for buying towers, added buying and placing of towers, updated tower assets (12h).

Markus: Worked on enemy damage with Alfred. Firing rate for towers, tower placement restrictions, pathfinding fixes (12h)

Alfred: Added hit detection for projectiles and enemies, function for enemy take damage from projectiles, testing. (8h)

Axel: Implemented the map creator which allows users to create maps of their own (12h)

Week 5:

David: Added upgrading of tower, visual indicator for upgrade, income and damage over time for poison tower (8h).

Markus: Implemented the missing towers and projectiles. Added new assets, Implemented waves and enemy logic, enemy death sound(12h)

Alfred: Testing bomb tower AoE implementation (unsuccessful), added freeze for ice tower. (8h)

Axel: Implemented the main menu that is shown when starting the game. (8h)

Week 6:

David: Added ending to game, various bugfixes, balancing and documentation updates (14h).

Markus: Documentation, wave transitions, bug fixing. (6h)

Alfred: Documentation, added immunity for stealth and boss enemy, small modifications. (6h)

Axel: Added instructions and fixed bugs in the map creator + documentation. (6h)